PKI Component API Reference

Verified against source code provided October 20, 2026 | Offline Root CA • Admin Workstation • Intermediate CA

Scope: exact function signatures and concise behavior. “Placeholder” means the function exists in source code but contains only pass; it is not operational. File-level constants and imported functions are not counted as new definitions.

Offline Root CA · common.py

Exact prototype	Purpose	Status
`run_command(command)`	Run a command, stringify arguments, and propagate subprocess errors.	Implemented

Offline Root CA · root_ca_setup.py

Exact prototype	Purpose	Status
`run_command(command)`	Run a subprocess command; locally redefines the imported utility.	Implemented
`create_ca_directories(ca_dir)`	Create certs, crl, csr, newcerts, private, and issued directories.	Implemented
`initialize_ca(ca_dir)`	Initialize index.txt, serial (1000), and crlnumber (01) if absent.	Implemented
`generate_root_config(root_dir, default_md=None)`	Write root OpenSSL configuration and certificate extension profiles.	Implemented
`generate_key(key_path, algorithm, options=None)`	Generate a private key unless the target file already exists.	Implemented
`create_root_certificate(root_dir)`	Create a self-signed root certificate unless it exists.	Implemented
`setup(mode="classical")`	Create directories, initialize database, generate key/config/root certificate.	Implemented

Offline Root CA · root_ca_operation.py

Exact prototype	Purpose	Status
`import_csr(csr_name, source_dir="/mnt/hgfs/transfer", dest_dir="/home/gxpics/aws-pki-root/rootCA/csr")`	Verify CSR signature and copy from shared media; reject overwrite.	Implemented
`sign_intermediate(intermediate_name, source_dir=root_dir / "csr", dest_dir=root_dir / "issued")`	Sign intermediate CSR with root CA and store issued certificate.	Implemented
`verify_intermediate(intermediate_cert_file, root_cert_dir=root_dir / "certs", intermediate_cert_dir=root_dir / "issued")`	Verify intermediate certificate against the root certificate.	Implemented
`revoke_certificate(intermediate_name, cert_dir=root_dir / "issued")`	Revoke certificate as superseded, update CA database, regenerate CRL.	Implemented
`generate_crl(crl_dir=root_dir / "crl")`	Generate root CA CRL and return output path.	Implemented
`archive_certificate(intermediate_name, source_dir=root_dir / "issued", archive_dir=root_dir / "archive")`	Move issued certificate into serial-numbered archive; reject collision.	Implemented
`export_to_media(source_path, transfer_dir)`	Copy a file to transfer directory (development workflow).	Implemented
`export_cert(cert_name, source_dir=None, transfer_dir="/mnt/hgfs/transfer")`	Validate certificate format and export via transfer directory.	Implemented
`main()`	Manual entry point; operation calls are currently commented out.	Implemented

Admin Workstation · admin_operation.py

Exact prototype	Purpose	Status
`download_csr(hostname, username, key_path, remote_csr_path, local_dir)`	Download CSR via authenticated SSH/SFTP; reject local overwrite.	Implemented

`export_to_media(source_path, transfer_dir)`	Copy a file to transfer directory; reject overwrite (development).	Implemented
`import_from_media(cert_name, transfer_dir, destination_dir, overwrite=False)`	Copy certificate from media to admin storage; overwrite only if enabled.	Implemented
`upload_certificates(hostname, username, key_path, local_cert_dir, remote_cert_dir, cert_names)`	Upload named certificates over SFTP; reject remote overwrite.	Implemented
`main()`	Run configured CSR download/export and certificate import/upload.	Implemented

Intermediate CA · intermediate_setup.py

Exact prototype	Purpose	Status
`run_command(command)`	Execute subprocess command and propagate errors.	Implemented
`create_ca_directories(ca_dir)`	Create CA working directories.	Implemented
`initialize_ca(ca_dir)`	Initialize index, serial, CRL counter, and unique_subject=no attribute.	Implemented
`generate_intermediate_config(ca_dir, ca_name, default_md=None)`	Write intermediate CA OpenSSL configuration.	Implemented
`generate_key(key_path, algorithm, options=None)`	Generate intermediate private key unless it exists.	Implemented
`generate_intermediate_csr(ca_dir, ca_name)`	Generate intermediate CSR unless it exists.	Implemented
`setup(mode="classical")`	Initialize intermediateCA1 and intermediateCA2, including keys and CSRs.	Implemented
`main()`	Invoke setup in classical mode.	Implemented

Intermediate CA · intermediate_operation.py

Exact prototype	Purpose	Status
`run_command(command)`	Execute subprocess command and propagate errors.	Implemented
`verify_intermediate_cert(ca_dir, ca_name)`	Verify signed intermediate certificate using rootCA.crt.	Implemented
`export_csr(ca_dir, ca_name, export_dir)`	Copy CSR to specified directory (local transfer workflow).	Implemented
`import_certificate()`	Intended to import signed intermediate certificate.	Placeholder
`install_root_certificate()`	Intended to install trusted root certificate.	Placeholder
`verify_key_pair()`	Intended to confirm certificate matches private key.	Placeholder
`create_ca_chain()`	Intended to construct CA certificate chain.	Placeholder
`sign_server_certificate()`	Intended to issue a server certificate.	Placeholder
`revoke_certificate()`	Intended to revoke an intermediate-issued certificate.	Placeholder
`generate_crl()`	Intended to generate intermediate CA CRL.	Placeholder
`main()`	Invoke export_csr for intermediateCA1 to /mnt/transfer.	Implemented

Implementation notes

- The offline Root CA and Intermediate CA each have separate run_command() definitions; root_ca_setup.py shadows the function imported from common.py.
- The offline Root CA revoke_certificate() invokes generate_crl() automatically; publishing the CRL to relying parties is not implemented in these files.
- Intermediate CA operational functions marked Placeholder are present but contain only pass.
- admin_operation.py main() currently sets intermediate_ca_name to intermediateCA2 while the download CSR parent path names intermediateCA1; review before running.
- The Root CA operation import_csr() validates CSR structure/signature but does not implement requester authorization or approval.